



Czestochowa
University
of Technology



Faculty of Computer Science
and Artificial Intelligence

SCRIPTING LANGUAGES IN WEB APPLICATIONS

L05 – Introduction to PHP



Web Programming Step by Step

Lecture 6-B

Introduction to PHP

Reading: 5.1 - 5.2

Except where otherwise noted, the contents of this presentation are Copyright 2010 Marty Stepp and Jessica Miller.



5.1: Server-Side Basics

- 5.1: Server-Side Basics
- 5.2: PHP Basic Syntax
- 5.3: Embedded PHP
- 5.4: Advanced PHP Syntax

URLs and web servers

`http://server/path/file`

- usually when you type a URL in your browser:
 - your computer looks up the server's IP address using DNS
 - your browser connects to that IP address and requests the given file
 - the web server software (e.g. Apache) grabs that file from the server's local file system, and sends back its contents to you
- some URLs actually specify *programs* that the web server should run, and then send their output back to you as the result:

`https://webster.cs.washington.edu/cse190m/quote.php`

- the above URL tells the server `webster.cs.washington.edu` to run the program `quote2.php` and send back its output

Server-Side web programming



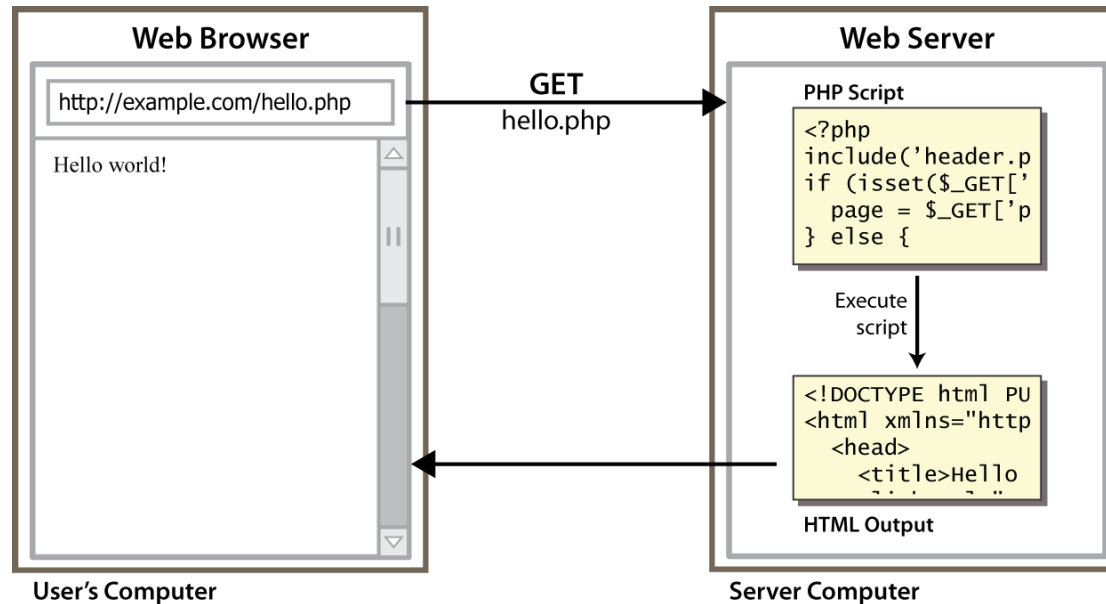
- server-side pages are programs written using one of many web programming languages/frameworks
 - examples: [PHP](#), [Java/JSP](#), [Ruby on Rails](#), [ASP.NET](#), [Python](#), [Perl](#)
- the web server contains software that allows it to run those programs and send back their output
- each language/framework has its pros and cons
 - we use PHP for server-side programming in this textbook

What is PHP? (5.1.2)

- **PHP** stands for "PHP Hypertext Preprocessor"
- a server-side scripting language
- used to make web pages dynamic:
 - provide different content depending on context
 - interface with other services: database, e-mail, etc
 - authenticate users
 - process form information
- PHP code can be embedded in XHTML code



Lifecycle of a PHP web request (5.1.1)



- browser requests a `.html` file (**static content**): server just sends that file
- browser requests a `.php` file (**dynamic content**): server reads it, runs any script code inside it, then sends result across the network
 - script produces output that becomes the response sent back

Why PHP?

There are many other options for server-side languages: Ruby on Rails, JSP, ASP.NET, etc. Why choose PHP?

- **free and open source:** anyone can run a PHP-enabled server free of charge
- **compatible:** supported by most popular web servers
- **simple:** lots of built-in functionality; familiar syntax
- **available:** installed on UW's servers (Dante, Webster) and most commercial web hosts
- **well-documented:** type `php.net/functionName` in browser Address bar to get docs for any function

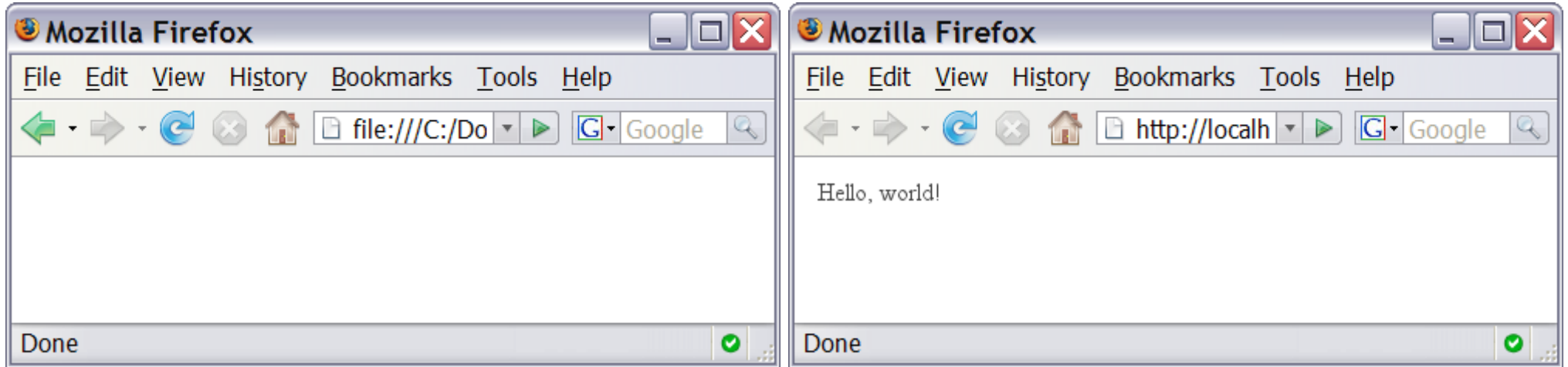
Hello, World!

The following contents could go into a file `hello.php`:

<pre><?php print "Hello, world!"; ?></pre>	<i>PHP</i>
Hello, world!	<i>output</i>

- a block or file of PHP code begins with `<?php` and ends with `?>`
- PHP statements, function declarations, etc. appear between these endpoints

Viewing PHP output



- you can't view your **.php** page on your local hard drive; you'll either see nothing or see the PHP source code
- if you upload the file to a PHP-enabled web server, requesting the **.php** file will run the program and send you back its output

5.2: PHP Basic Syntax

- 5.1: Server-Side Basics
- **5.2: PHP Basic Syntax**
- 5.3: Embedded PHP
- 5.4: Advanced PHP Syntax

Console output: **print** (5.2.2)

```
print "text";  
  
print "Hello, World!\n";  
print "Escape \"chars\" are the SAME as in Java!\n";  
  
print "You can have  
line breaks in a string.";  
  
print 'A string can use "single-quotes". It\'s cool!';
```

PHP

PHP

Hello, World! Escape "chars" are the SAME as in Java! You can have line breaks in a string. A string can use "single-quotes". It's cool!

output

- some PHP programmers use the equivalent **echo** instead of **print**

Arithmetic operators (5.2.4)

- + - * / %
 . ++ --
 = += -= *= /= %= .=
• many operators auto-convert types: 5 + "7" is 12

Variables (5.2.5)

```
$name = expression;
```

PHP

```
$user_name = "PinkHeartLuvr78";  
$age = 16;  
$drinking_age = $age + 5;  
$this_class_rocks = TRUE;
```

PHP

- names are case sensitive; separate multiple words with _
- names always begin with \$, on both declaration and usage
- implicitly declared by assignment (type is not written; a "loosely typed" language)

Types (5.2.3)

- basic types: `int`, `float`, `boolean`, `string`, `array`, `object`, `NULL`
 - test what type a variable is with `is_type` functions, e.g. `is_string`
 - `gettype` function returns a variable's type as a string (not often needed)
- PHP `converts between types automatically` in many cases:
 - `string` → `int` auto-conversion on + (`"1" + 1 == 2`)
 - `int` → `float` auto-conversion on / (`3 / 2 == 1.5`)
- type-cast with (`type`):
 - `$age = (int) "21";`

Comments (5.2.7)

```
# single-line comment

// single-line comment

/*
multi-line comment
*/
```

PHP

- like Java, but **#** is also allowed
 - a lot of PHP code uses **#** comments instead of **//**
 - we recommend **#** and will use it in our examples

String type (5.2.6)

```
$favorite_food = "Ethiopian";
print $favorite_food[2];
```

h

PHP

- zero-based indexing using bracket notation
- string concatenation operator is **.** (period), not **+**
 - `5 + "2 turtle doves" === 7`
 - `5 . "2 turtle doves" === "52 turtle doves"`
- can be specified with `" "` or `' '`

Interpreted strings

```
$age = 16;  
print "You are " . $age . " years old.\n";  
print "You are $age years old.\n";    # You are 16 years old.
```

PHP

- strings inside " " are **interpreted**
 - variables that appear inside them will have their values inserted into the string
- strings inside ' ' are *not* interpreted:

```
print 'You are $age years old.\n';    # You are $age years old.\n
```

PHP

- if necessary to avoid ambiguity, can enclose variable in {}:

```
print "Today is your $ageth birthday.\n";    # $ageth not found  
print "Today is your {$age}th birthday.\n";
```

PHP

for loop (same as Java) (5.2.9)

```
for (initialization; condition; update) {  
    statements;  
}
```

PHP

```
for ($i = 0; $i < 10; $i++) {  
    print "$i squared is " . $i * $i . ".\n";  
}
```

PHP

if/else statement

```
if (condition) {  
    statements;  
} elseif (condition) {  
    statements;  
} else {  
    statements;  
}
```

PHP

- NOTE: although **elseif** keyword is much more common, **else if** is also supported

while loop (same as Java)

```
while (condition) {  
    statements;  
}
```

PHP

```
do {  
    statements;  
} while (condition);
```

PHP

- **break** and **continue** keywords also behave as in Java

Slides licensed under a Creative Commons (CC BY-NC-ND 4.0)

Attribution-NonCommercial-NoDerivatives 4.0 International License

YOU CAN SHARE UNDER THE FOLLOWING CONDITIONS

(share, reproduce, distribute, transmit, execute and play with any means and in any format)

Attribution*

You must give appropriate credit, provide a link to the license, and indicate if changes were made.

You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.

Non Commercial

You may not use the material for commercial purposes.

No derivatives

If you remix, transform, or build upon the material, you may not distribute the modified material.

No additional restrictions

You may not apply legal terms or technological measures that legally restrict others from doing anything the license permits.

DISCLAIMER

the slides may contain copyright-licensed third part materials



Częstochowa
University
of Technology



Faculty of Computer Science
and Artificial Intelligence